Week 4: Advanced Threat Detection & Web Security Enhancements

Overview

This project implements advanced security mechanisms to protect web applications from common cyber threats such as brute-force attacks, unauthorized API access, and script injection. It includes real-time intrusion detection, API security hardening, and implementation of modern web security standards.

Features

1. Intrusion Detection & Monitoring

- Integrated **Fail2Ban / OSSEC**
- Real-time monitoring of server logs
- Automatic detection of:
  - Multiple failed login attempts
  - Suspicious IP activity
- Alerts and blocking mechanisms for malicious behavior

2. API Security Hardening

Prevents excessive requests using `express-rate-limit`.

```
const rateLimit = require("express-rate-limit");
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP
  message: "Too many requests, please try again later."
});
```

- ***CORS Configuration***

Restricts access to trusted origins only.

```
const cors = require("cors");
app.use(cors({
  origin: "https://yourdomain.com",
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: true
}));
```

- ***API Authentication***

API Key Example:

```
const apiKeyMiddleware = (req, res, next) => {
  const apiKey = req.headers["x-api-key"];
 if (apiKey && apiKey === process.env.API_KEY) {
    next();
  } else {
    res.status(403).json({ message: "Unauthorized" });
  }
};
app.use("/api/secure", apiKeyMiddleware);
```

OAuth (Conceptual):

- Use libraries like `passport.js`
- Supports secure login via third-party providers

3. Security Headers & CSP

Helmet Setup

```
const helmet = require("helmet");
app.use(helmet());
```

- ***Content Security Policy (CSP)***

```
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'", "trusted-cdn.com"]
  }
}));
```

- ***HSTS (HTTPS Enforcement)***

```
app.use(helmet.hsts({
  maxAge: 31536000,
  includeSubDomains: true
}));
```

Technologies Used

- Node.js
- Express.js
- express-rate-limit
- Helmet
- CORS
- Fail2Ban / OSSEC

Project Structure

```
1. project-root/
2. Config/
3. Middlewares/
4. Routes/
5. Security/
6. App.js
7. package.json
```

Installation & Setup

1. Clone Repository

```
git clone https://github.com/your-username/your-repo.git
```

2. Install Dependencies

```
npm install
```

3. Environment Variables

Create a `.env` file:

```
PORT=3000
API_KEY=your_secret_key
```

4. Run Server

```
npm start
```